Declaration

```
template<typename Lockable,typename Clock,
    typename Duration, typename Predicate>
bool wait_until(
    Lockable& lock,
    std::chrono::time_point<Clock,Duration> const& absolute_time,
    Predicate pred);
```

Preconditions

The expression `pred()` shall be valid, and shall return a value that's convertible to `bool`. `Lockable` meets the `Lockable` requirements, and `lock` owns a lock.

Effects

As-if

```
while(!pred())
{
    if(wait_until(lock,absolute_time)==std::cv_status::timeout)
        return pred();
}
return true;
```

Returns

`true` if the most recent call to `pred()` returned `true`, `false` if a call to `Clock::now()` returned a time equal to or later than the time specified by `absolute_time`, and `pred()` returned `false`.

Note

The potential for spurious wakeups means that it's unspecified how many times `pred` will be called. `pred` will always be invoked with the mutex referenced by `lock` locked, and the function shall return if (and only if) an evaluation of `(bool)pred()` returns `true` or `Clock::now()` returns a time equal to or later than `absolute_time`. There's no guarantee as to